Running SIMPLE on CIPRB's own server

A step by step setup guide

Integrated Digital M&E System
Centre for Injury Prevention and Research, Bangladesh
Handover document 8 · November 2026

A step by step guide, from an empty machine to a working system with field data arriving.

This is written for the person who will administer the server. It assumes comfort with a Linux command line and nothing else. Every step says what to do, and then how to check it actually worked before moving on.

Do not skip the checks. Most of the ways this goes wrong are silent. The system will look fine and simply not receive any data.

Contents

Step 0	Decide whether this is possible, before anything else
Step 1	Collect what you need
Step 2	Prepare the server
Step 3	Get the code
Step 4	Write the configuration
Step 5	Start the database and the application
Step 6	Put Nginx and HTTPS in front
Step 7	Create the login accounts
Step 8	Bring the existing data across
Step 9	Deploy the forms in your own KoboToolbox account
Step 10	Test end to end with a real submission
Step 11	Set up the scheduled jobs
Step 12	Set up backups
Step 13	Point the last settings at your own domain
	Day to day operation
	Rules that must not be broken
	Troubleshooting
	Final checklist

Step 0. Decide whether this is possible

Read this before buying or configuring anything.

Field data does not arrive because the server goes and fetches it. It arrives because KoboToolbox pushes it in. Every time a field worker presses submit, Kobo's servers in Europe make an HTTPS request to your server:

```
POST https://simple.ciprb.org/webhook/programs/form/spondon_clinic_visit_v1/
```

For that to work, **KoboToolbox has to be able to reach your server from the public internet.**

So the server must have:

- a public IP address, or a port forward from one
- a domain or subdomain pointing at that address, for example `simple.ciprb.org`
- ports 80 and 443 open inbound, **from anywhere**, not only from CIPRB offices
- a TLS certificate, which is free from Let's Encrypt

A machine sitting inside the CIPRB office network, behind the office firewall, with only a private address such as `192.168.1.50`, **will not work.** The dashboard would open fine from inside the office, and no field submission would ever arrive. Nothing would show an error. This is the single most likely way to get this wrong.

Restricting inbound access to CIPRB office IP addresses has the same effect. It blocks KoboToolbox.

If CIPRB IT cannot provide a public domain and certificate for an in-office machine, the usual answer is a small cloud server. The requirement is identical either way. It is only a question of where the machine sits.

Decide this first. Everything below assumes the answer is yes.

Step 1. Collect what you need

Gather all of these before starting. Several cannot be obtained later.

1a. The server

	Minimum	Comfortable
CPU	2 cores	2 cores
RAM	2 GB	4 GB
Disk	20 GB	40 GB

Operating system: Debian 12, or Ubuntu 22.04 or later. The application image is built on Debian, so these are the tested combinations.

RAM is the number that matters. The web process has been seen growing from about 80 MB to about 520 MB between restarts, and generating a monthly report starts a headless Chromium browser that wants roughly another gigabyte while it runs. At 2 GB the system works, but report generation will be tight. 4 GB removes the worry.

1b. The domain and certificate

A subdomain such as `simple.ciprb.org`, with a DNS A record pointing at the server's public IP address.

1c. The secrets, from the current system

These come from the environment of the system running today. **Get them before it is switched off.**

What	Why it cannot be regenerated
`FERNET_KEY`	**The critical one.** Three patient-identifying fields in the fistula and MPDSR modules are encrypted with this key. If you start with a new key, every one of those existing records becomes permanently unreadable. There is no recovery, no workaround and no support request that will fix it
`KOBO_API_TOKEN`	The KoboToolbox API token
`KOBO_WEBHOOK_SECRET`	The shared secret that proves an incoming webhook really came from Kobo
`KOBO_ASSET_UID_*`	Eight form identifiers. `.env.example` lists them all
`GROQ_API_KEY`	Only if the AI written report narrative is wanted
`TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_IDS`	Only if Telegram alerts are wanted

`SECRET_KEY` is the exception. Generate a fresh one. Do not reuse the old.

1d. The database file

A `pg_dump` file, which will be supplied to you. It is taken on the day of the move, so ask for it then rather than in advance. See Step 8.

1e. A KoboToolbox account, and the form files

Your own KoboToolbox account, and its API token from the account settings. You will deploy the forms there yourself in Step 9.

You will also be given the fourteen forms as XLSForm files, in a folder called `CIPRB_Kobo_Forms_XLSForm`. Without these, Step 9 cannot be done and no field data will ever arrive.

Step 2. Prepare the server

Log in to the new server as a user with `sudo`.

2a. Update the system

```
sudo apt update && sudo apt upgrade -y
```

2b. Set the timezone

Do this before setting up the scheduled jobs in Step 11, or they will run at the wrong hour.

```
sudo timedatectl set-timezone Asia/Dhaka
```

Check:

```
date
```

It should print Bangladesh time.

2c. Install Docker

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

Log out and back in, so the group change takes effect.

Check:

```
docker --version
docker compose version
```

Both must print a version. Docker Engine should be 24 or later.

Everything the application needs, Python 3.12, Node 20, Chromium and all its libraries, is inside the image. Nothing else has to be installed on the server.

> **If CIPRB policy forbids Docker**, the application can be installed directly: Python 3.12 with a virtual environment, PostgreSQL 17 or later, Node 20 to build the frontend once, Gunicorn, and `python -m playwright install --with-deps chromium` for the report renderer. It is more work to set up and more work to keep consistent, and it no longer matches what has been tested. Use Docker unless there is a policy reason not to.

2d. Install Nginx and Certbot

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

2e. Open the firewall

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 22/tcp
sudo ufw enable
```

Check, from a machine outside the CIPRB network:

```
curl -I http://simple.ciprb.org
```

You should get a response from Nginx, even an error page. If it times out, the DNS record or the firewall or the port forward is wrong. **Fix this before continuing.** Step 0 explains why nothing works without it.

Step 3. Get the code

```
sudo mkdir -p /srv/simple
sudo chown $USER:$USER /srv/simple
git clone https://github.com/rafijahiin/simple-handover.git /srv/simple
cd /srv/simple
```

Check:

```
ls
```

You should see `manage.py`, `Dockerfile`, `requirements.txt` and the application folders.

Step 4. Write the configuration

```
cp .env.example .env
nano .env
```

Fill it in as below. Replace every `<...>` with a real value.

Required. Nothing works without these.

```
DJANGO_SETTINGS_MODULE=spondon.settings.production
SECRET_KEY=<a fresh random string, see below>
DATABASE_URL=postgresql://simple:<db password>@db:5432/simple
ALLOWED_HOSTS=simple.ciprb.org
FERNET_KEY=<the EXISTING key from the current system>
SIMPLE_PUBLIC_URL=https://simple.ciprb.org
```

Generate `SECRET_KEY` with:

```
python3 -c "import secrets; print(secrets.token_urlsafe(64))"
```

`FERNET_KEY` must be the existing one. Read Step 1c again if there is any doubt.

`SIMPLE_PUBLIC_URL` must be changed. Its built-in default is the previous host's address, and leaving it would make the system generate links pointing back at a server you do not control.

`ALLOWED_HOSTS` is also where the CSRF trusted origins come from, derived automatically. If the dashboard later refuses logins with a CSRF error, this is the variable to check.

Required for the system to do its actual job.

```
KOBO_API_TOKEN=<from the current system>
KOBO_SERVER_URL=https://kf.kobotoolbox.org
KOBO_WEBHOOK_SECRET=<from the current system>
KOBO_ASSET_UID_MPDSR=<from the current system>
KOBO_ASSET_UID_FISTULA=<from the current system>
KOBO_ASSET_UID_ACTIVITY=<from the current system>
KOBO_ASSET_UID_BASELINE=<from the current system>
KOBO_ASSET_UID_CLIENT_REG=<from the current system>
```

`KOBO_WEBHOOK_SECRET` deserves a note. The webhook **fails closed**: if this variable is empty, every incoming request is rejected with 403 and no field data is accepted at all. This is deliberate. An unprotected webhook on a public address would let anyone on the internet write records into CIPRB's database.

Optional. Leave a line out entirely to switch the feature off.

```
GROQ_API_KEY=<only for the AI written report narrative>
TELEGRAM_BOT_TOKEN=<only for submission gap alerts>
TELEGRAM_CHAT_IDS={"PHD": "-100...", "Bondhu": "-100...", "CIPRB": "-100...", "UNFPA": "-100..."}
KOBO_DELETION_SYNC=all
ENABLE_RECON_LOOP=1
ENABLE_CSV_RESYNC_LOOP=1
```

Redis is optional and is not used in production today. Without `REDIS_URL`, Django caches in the web process's own memory, which is correct and sufficient at one worker.

Protect the file

```
chmod 600 .env
```

It holds the encryption key. Anyone who can read it can read patient data.

Step 5. Start the database and the application

`docker-compose.yml` is already in the repository. Open it and change one line, `POSTGRES_PASSWORD`, to the same password you put in `DATABASE_URL` in Step 4:

```
nano docker-compose.yml
```

Note `127.0.0.1:8080:8080`. The application is deliberately **not** exposed to the internet directly. Nginx sits in front of it and holds the certificate.

Build and start. The first build takes ten to twenty minutes, because it downloads Chromium.

```
docker compose up -d --build
```

The container runs database migrations automatically on every start. This is safe and can be repeated.

Check:

```
curl -i http://127.0.0.1:8080/api/health/
```

You want `HTTP/1.1 200 OK`. If not:

```
docker compose logs web --tail 50
```

The most common cause is `DATABASE_URL` not matching the password in `docker-compose.yml`. The application deliberately refuses to start without a working database rather than quietly falling back to a temporary one.

Step 6. Put Nginx and HTTPS in front

Create `/etc/nginx/sites-available/simple`:

```
server {
    listen 80;
    server_name simple.ciprb.org;

    # Photographs and scans from the field.
    client_max_body_size 25M;

    location / {
        proxy_pass         http://127.0.0.1:8080;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }
}
```

Three settings there are not decoration:

- `X-Forwarded-Proto` tells Django the request arrived over HTTPS. Without it, secure cookies misbehave and logins fail in confusing ways.
- `proxy_read_timeout 120s` because generating a monthly report with Chromium can take longer than Nginx's default 60 seconds.
- `client_max_body_size 25M` because field submissions carry photographs. Without it, uploads fail with a 413.

Enable it and get the certificate:

```
sudo ln -s /etc/nginx/sites-available/simple /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
sudo certbot --nginx -d simple.ciprb.org
```

Certbot edits the file to add the certificate and the redirect from HTTP to HTTPS, and installs a timer that renews it automatically.

Check, from outside the CIPRB network:

```
curl -i https://simple.ciprb.org/api/health/
```

`HTTP/2 200`. Then open `https://simple.ciprb.org` in a browser. The dashboard login page should appear with a valid padlock.

Step 7. Create the login accounts

Only for a new installation. If you are restoring an existing database in Step 8 the accounts come with it, so skip this step.

Create the first account, then give it the `developer` role, which is the one with full access:

```
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py update_user_role you@ciprb.org --role developer --org CIPRB
```

`createsuperuser` asks for an email, a full name and an organisation. The organisation must be one of `CIPRB`, `PHD` or `Bandhu`. The second command is not optional: a new account is created with the `manager` role, and being a Django superuser does not by itself grant access, because this application checks the role, not the superuser flag.

The seven roles are `developer`, `supervisor`, `org_lead`, `manager`, `field_staff`, `ciprb_baseline` and `focal`.

Check: log in at `https://simple.ciprb.org`.

Create everyone else from inside the application once you are in.

> There is also a `seed_users` command that creates the original project's standard account set. It is gated on `SEED_DB=1` **and** on a separate password variable for each account, listed at the bottom of `.env.example`. An account whose password variable is blank is skipped with a warning, so setting `SEED_DB=1` on its own creates nobody. For a new installation `createsuperuser` is simpler, and it does not recreate the previous team's email addresses.

Step 8. Bring the existing data across

The order of these matters. Do not reorder them.

8a. Obtain the database file

You will be given a `pg_dump` file, `simple-backup.dump`. Copy it onto the server, for example into `/srv/simple`.

Records belonging to the implementing partners, PHD and Bandhu, have been removed from it before handover, so those two dashboards will be empty on the new system. The CIPRB data, meaning the maternal and perinatal death surveillance, near miss, fistula, safe surgery, baseline and training records, is all present. This is deliberate, not a failed restore.

The file contains patient-identifying information. Handle it accordingly, and delete your copy once the restore is verified.

8b. Confirm FERNET_KEY is already in place

Check that the `FERNET_KEY` in `/srv/simple/.env` is the **existing** key, not a new one. If it is wrong, every encrypted patient field in the restored data becomes permanently unreadable. Confirm this now, before restoring.

8c. Restore

Stop the application first. It runs migrations and background jobs on start, and those will fight a restore that is replacing the same tables underneath them.

```
docker compose stop web
docker compose cp simple-backup.dump db:/tmp/
```

Restore into a freshly created database rather than over the top of the one the application has already migrated. `--single-transaction` means a failure part way through leaves nothing behind, instead of a half-loaded database that looks fine until someone opens the wrong page.

```
docker compose exec db dropdb   -U simple --if-exists simple
docker compose exec db createdb -U simple simple
docker compose exec db pg_restore -U simple -d simple   --no-owner --no-privileges --single-transaction /tmp/simple-backup.dump
docker compose start web
```

Check:

```
docker compose exec web python manage.py shell -c \
  "from submissions.models import KoboSubmission; print(KoboSubmission.objects.count())"
```

You should get a non-zero number. Then open the dashboard and confirm the CIPRB figures look right, and that a fistula or MPDSR record displays its patient name correctly rather than as unreadable text. If the names are unreadable, `FERNET_KEY` is wrong. **Stop and fix it before going further**, and do not re-save any of those records while the key is wrong.

Compare the CIPRB totals against the current dashboard. They should match. The PHD and Bandhu figures will not, for the reason in 8a.

8d. Rotate the encryption key, afterwards

Do this once the restore is verified working, and not before. The key has been in use on another system, so it should not stay in service indefinitely on yours.

Treat it as a separate piece of maintenance, on a quiet day, with a fresh backup taken first. Not part of go-live.

```
docker compose exec web python manage.py rotate_fernet_key
```

That is a **dry run**. It reports what it would re-encrypt and changes nothing. Read the output, then apply it with `--commit`:

```
docker compose exec web python manage.py rotate_fernet_key --commit
```

Put the old key in `FERNET_KEY_FALLBACKS` and the new one in `FERNET_KEY` before running with `--commit`, so the command can read the old records and write them back under the new key.

It works by reading with the old key and writing with the new one. Do **not** rotate by editing `FERNET_KEY` in `.env` and restarting. That leaves every existing record encrypted under a key the system no longer has.

Step 9. Deploy the forms in your own KoboToolbox account

This is the step that actually moves the system. Until it is done, field submissions still arrive at the old server, no matter how well the new one works.

You have been given the forms as XLSForm files, in `CIPRB_Kobo_Forms_XLSForm`. There are fourteen. Import each one into your own KoboToolbox account and deploy it.

9a. Import and deploy

In KoboToolbox: **New project**, then **Import an XLSForm**, choose the file, then **Deploy**. Repeat for all fourteen.

Keep the files as they are. Do not rename them and do not edit the `settings` sheet. The `form_id` inside each file is what tells this system which handler to use, so if it changes, submissions arrive and are rejected as unknown.

9b. Set the REST Service on each form

The forms do not all use the same endpoint. Twelve go to one address and the two baseline surveys go to another. Sending a form to the wrong one means its submissions arrive and are rejected as unknown.

For each form: **Settings**, then **REST Services**, then add one.

Field	Value
Endpoint URL	from the table below
Method	POST
Header name	`Authorization`
Header value	`Token <your KOBO_WEBHOOK_SECRET>`

Replace `simple.ciprb.org` with your own domain. The last part of each URL is the form's own id, so the twelve are all different.

Excel file	Endpoint URL
`CIPRB-1_Fistula_Question_Bank.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_fistula_questions_v1/`
`CIPRB-1b_Fistula_Campaign.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_fistula_campaign_v1/`
`CIPRB-2_MPDSR_Form_01_Community_Maternal.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_mpdsr_community_maternal_v1/`
`CIPRB-3_MPDSR_Form_02_Community_Neonatal.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_mpdsr_community_neonatal_v1/`
`CIPRB-4_MPDSR_Form_04_Facility_Maternal.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_mpdsr_facility_maternal_v1/`
`CIPRB-5_MPDSR_Form_05_Facility_Neonatal.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_mpdsr_facility_neonatal_v1/`
`CIPRB-6_Social_Autopsy.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_social_autopsy_v1/`
`CIPRB-7_Notification_Slip_01.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_notification_slip_01_v1/`
`CIPRB-8_Notification_Slip_02.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_notification_slip_02_v1/`
`CIPRB-9_Maternal_Near_Miss.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_near_miss_v1/`
`CIPRB-10_MPDSR_Response_Plan.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_mpdsr_response_plan_v1/`
`CIPRB-11_Safe_Surgery_Checklist.xlsx`	`https://simple.ciprb.org/webhook/programs/form/ciprb_safe_surgery_v1/`
`CIPRB_Baseline_FSW.xlsx`	`https://simple.ciprb.org/webhook/kobo/`
`CIPRB_Baseline_Hijra.xlsx`	`https://simple.ciprb.org/webhook/kobo/`

The two baseline surveys are the exception. They are routed by the form id inside the file, so they share the one address.

9c. Collect the new asset UIDs

Deploying in your own account gives every form a **new asset UID**. The old ones belong to the previous account and are no longer valid.

The UID is in the project's URL, the `a...` part:

```
https://kf.kobotoolbox.org/#/forms/aXXXXXXXXXXXXXXXXXXXX/summary
                                   ^^^^^^^^^^^^^^^^^^^^
```

Write down the UID of each form. You need them in 9d and 9e.

9d. Put the new UIDs in `.env`

Routing does not need these, because it goes by the form id. They are needed for everything that reads from or writes back **into** KoboToolbox.

```
KOBO_API_TOKEN=<your own token, from your KoboToolbox account settings>
KOBO_TOKEN=<the same value>
KOBO_WEBHOOK_SECRET=<a secret you choose, the same one used in 9b>

KOBO_ASSET_UID_FISTULA_QB=<CIPRB-1 Fistula Question Bank>
KOBO_ASSET_UID_MPDSR_ACTION=<CIPRB-10 MPDSR Response Plan>
KOBO_ASSET_UID_MPDSR=<CIPRB-2 Community Maternal>
KOBO_ASSET_UID_FISTULA=<CIPRB-1 Fistula Question Bank, same as above>
KOBO_ASSET_UID_BASELINE=<CIPRB Baseline FSW>
```

The first two matter most. When a fistula case or an MPDSR action is created, the system issues the official ID and writes it back into the Kobo submission, so the paper and the screen agree. They are also the two forms that receive the lookup files in 9f. If these still hold the old account's UIDs, the system tries to edit forms you do not own, and both the write-back and the lookup files fail.

The token and the webhook secret are your own. You do not need the previous ones.

Restart after editing: `docker compose up -d`

9e. Tell the system your new asset UIDs

Everything that writes back **into** KoboToolbox needs to know your UIDs: the case and action ID write-back, the lookup files in 9f, and the reconciliation check. Until this is done they still point at the account the system was first deployed from.

There is a command that does it for you. It reads your KoboToolbox account, matches each form by the form id inside the XLSForm, and fills in the UIDs. No typing, and no chance of pasting the wrong UID against the wrong form.

Make sure `KOBO_API_TOKEN` in `.env` is your own token, then:

```
docker compose exec web python manage.py sync_kobo_uids
```

That is a dry run. It prints every form, the UID it found, and what it would change. Read it, then write the values:

```
docker compose exec web python manage.py sync_kobo_uids --apply
```

Expect to see all fourteen forms listed. If one says **NOT FOUND**, it has not been deployed in your account yet, or the `settings` sheet in its Excel file was edited and the form id no longer matches. Deploy it, or restore the original file, and run the command again.

> If you would rather do it by hand, the table is `indicators_koboformmapping`, one row per form, column `kobo_asset_uid`, keyed on `form_slug`. The command is the safer route.

9f. Upload the two lookup files

Two forms read a CSV of existing records so the field worker can pick one instead of typing it again. Those CSVs are generated from your database and attached to the Kobo form as media. New projects have no media, so this has to be done once, after 9d.

```
docker compose exec web python manage.py export_fistula_clients --upload
docker compose exec web python manage.py export_mpdsr_actions --upload
```

`--upload` **is not optional.** Without it both commands only write the CSV locally and print a summary, which looks like success and attaches nothing.

They send to whichever UID step 9e put in the registry, so do 9e first.

The first builds `fistula_clients.csv` from the registered patients and attaches it to CIPRB-1. Without it, a field worker reaching the Diagnosed, Referred, Repaired or Rehabilitated stage cannot find the woman she registered, and the duplicate check at registration stops working.

The second builds `mpdsr_actions.csv` and attaches it to CIPRB-10. Without it, the committee cannot pick an existing action to update.

Both commands upload and redeploy the form, so open each one in KoboToolbox afterwards and confirm a new version was published.

Check: open CIPRB-1 in Enketo and confirm the patient dropdown lists the restored patients. If it is empty, the UID in 9d is wrong.

9g. About the old submissions

The historical records are already in the database you restored in Step 8, so the dashboard is complete from day one.

What does **not** come across is the raw submission history inside KoboToolbox. Your new forms start with zero submissions in Kobo, while the dashboard shows the full history. That is expected. If you ever need the original Kobo-side exports, ask for them before the old account is closed.

Step 10. Test end to end with a real submission

Do not declare the move finished until this passes.

1. On a phone with KoboCollect, open any form and submit one test record.
2. Watch the server receive it:

```
docker compose logs web --tail 50 -f
```

3. Open the dashboard. The record should appear in the approval queue.
4. Approve it, and confirm the count moves.
5. Delete the test record afterwards.

If nothing arrives, in this order:

- In KoboToolbox, open the form's REST Service and read its log. Kobo records the response it got. A 403 means `KOBO_WEBHOOK_SECRET` or the Authorization header is wrong. A timeout means Kobo cannot reach the server at all, which takes you back to Step 0.
- Confirm the endpoint URL has no typo and ends with a slash.
- Confirm `curl -i https://simple.ciprb.org/api/health/` works **from outside** the CIPRB network, not just from inside.

Step 11. Set up the scheduled jobs

Two jobs run on a schedule. They are not part of the web process and will not run unless cron is set up.

```
crontab -e
```

```
# Submission gap alerts, every morning at 09:00 Dhaka time.
0 9 * * *  cd /srv/simple && docker compose exec -T web python manage.py run_alerts

# The monthly reporting kit, on the 1st of each month at 02:00.
0 2 1 * *  cd /srv/simple && docker compose exec -T web python manage.py generate_monthly_hub
```

The `-T` matters. Without it the command fails under cron, because there is no terminal.

Check: run each once by hand and confirm it completes.

```
cd /srv/simple && docker compose exec -T web python manage.py run_alerts
```

Step 12. Set up backups

The database is the only thing here that cannot be rebuilt.

```
sudo mkdir -p /srv/backups
crontab -e
```

```
# Daily at 01:00, keeping 30 days.
0 1 * * * cd /srv/simple && docker compose exec -T db pg_dump -U simple simple | gzip > /srv/backups/simple-$(date +\%F).sql.gz
30 1 * * * find /srv/backups -name 'simple-*.sql.gz' -mtime +30 -delete
```

Two more things must be backed up, separately, and kept somewhere safe:

1. `FERNET_KEY`**.** A database backup without the key is **not a complete backup**. Restoring it would give you a database whose patient fields cannot be read.
2. **The** `mediafiles` **volume**, which holds uploaded photographs and scans.

```
docker run --rm -v simple_media:/data -v /srv/backups:/backup alpine \
  tar czf /backup/media-$(date +%F).tar.gz -C /data .
```

Test a restore into a scratch database before trusting any of this. An untested backup is a guess, not a backup.

Step 13. Point the last settings at your own domain

Three settings still carry the address of the server the system was previously hosted on. None of them stop it working, but they should point at your own domain.

1. `mpdsr/reconcile.py`**, lines 92 and 194.** The reconciliation check decides whether a Kobo webhook points at the right place by matching the endpoint against the previous host's name. Until this reads `SIMPLE_PUBLIC_URL`, that check will report your webhooks as misconfigured even when they are correct. **Change this one**, because a monitoring check that is always wrong gets ignored.
2. `programs/management/commands/build_no_report_form.py`**, line 34.** `APP_BASE` is the previous address, hardcoded. It only matters when rebuilding that one form.
3. `spondon/settings/production.py`**, line 35.** Appends a health-check hostname belonging to the previous host to `ALLOWED_HOSTS`. Harmless, just no longer meaningful.

Day to day operation

Task	Command, run from `/srv/simple`
See the logs	`docker compose logs web --tail 100 -f`
Restart	`docker compose restart web`
Stop	`docker compose down`
Start	`docker compose up -d`
Update to a new version	`git pull && docker compose up -d --build`
Open a Django shell	`docker compose exec web python manage.py shell`
Check disk space	`df -h` and `docker system df`
Reclaim disk from old images	`docker system prune -a`

Migrations run automatically on every start, so an update is just pull and rebuild.

Rules that must not be broken

1. Run exactly one web worker.

The production command is deliberately `gunicorn --workers 1 --threads 4`. Do not raise the worker count, and do not run two containers behind a load balancer. Five background jobs run as threads **inside** the web process:

Thread	Job
`kobo-deletion-sync`	Every six hours, reconciles records deleted in Kobo
`ciprb-recon-loop`	CIPRB reconciliation
`csv-resync-loop`	Keeps the Kobo pulldata CSV files current
`bandhu-id-writeback`	Writes server issued IDs back into Kobo
`client-csv-sync`	Pushes the client list back to Kobo

Two workers means two copies of each, racing one another. If the server needs more capacity, raise `--threads`, never `--workers`.

2. Never change `FERNET_KEY` by hand. Use `rotate_fernet_key`. Editing it makes every encrypted patient record permanently unreadable.

3. Never close inbound 443 to the public. It blocks KoboToolbox and stops all field data, silently.

4. Never leave `KOBO_WEBHOOK_SECRET` empty. The webhook then rejects everything, by design.

5. Records that disappear on their own are usually normal. KoboToolbox does not notify this system when a submission is deleted there. A sweep runs every six hours and reconciles, so a deletion made in Kobo takes six to twelve hours to disappear from the dashboard. It is recoverable from the `KoboWithdrawal` table.

The network access the server needs

Outbound HTTPS, which the firewall must allow:

Host	Used for
`kf.kobotoolbox.org` , `ee.kobotoolbox.org`	Reading form data and deploying forms
`api.groq.com`	AI report narrative, only if `GROQ_API_KEY` is set
`api.telegram.org`	Alerts, only if Telegram is configured
`fonts.googleapis.com`	Fonts for the rendered reports

Inbound, ports 80 and 443, from anywhere.

Troubleshooting

Symptom	Where to look
Container will not start	`docker compose logs web --tail 50`. Usually `DATABASE_URL` does not match the password in `docker-compose.yml`
400 Bad Request on every page	`ALLOWED_HOSTS` does not contain the domain being used
Login fails with a CSRF error	`X-Forwarded-Proto` missing from the Nginx config, or the domain missing from `ALLOWED_HOSTS`
Dashboard works, no field data arrives	Step 9 is incomplete, or Step 0 was never satisfied. Read the REST Service log inside KoboToolbox
Webhook returns 403	`KOBO_WEBHOOK_SECRET` does not match the `Authorization: Token ...` header in Kobo
Patient names show as unreadable text	Wrong `FERNET_KEY`. Restore the correct one. Do not re-save those records while the key is wrong
Report generation times out	`proxy_read_timeout` too low in Nginx, or the server is short of RAM for Chromium
Photograph upload fails with 413	`client_max_body_size` too small in Nginx
Records vanish without anyone deleting them	Normal. The Kobo deletion sweep. See rule 5 above
Numbers look wrong after a deploy	Check the approval queue first. Everything lands pending and does not count until approved

For anything not listed here, see `docs/handover/SIMPLE_Handover_2_Troubleshooting_Playbook.pdf`.

Final checklist

Work down this list. Do not treat the move as complete until every line is ticked.

Before starting

- [ ] Server has a public IP and a domain pointing at it
- [ ] Ports 80 and 443 reachable from outside the CIPRB network
- [ ] `FERNET_KEY` obtained from the current system and stored safely
- [ ] All other secrets collected from the current system
- [ ] Access to edit REST Services on every KoboToolbox form

Setup

- [ ] Timezone set to `Asia/Dhaka`
- [ ] Docker and Compose installed
- [ ] Code cloned, `.env` written, `chmod 600` applied
- [ ] `curl http://127.0.0.1:8080/api/health/` returns 200
- [ ] Nginx configured and certificate installed
- [ ] `curl https://simple.ciprb.org/api/health/` returns 200 **from outside**
- [ ] Dashboard opens in a browser with a valid padlock

Data

- [ ] `pg_dump` of the current database taken
- [ ] `FERNET_KEY` confirmed correct **before** restoring
- [ ] Database restored, record counts match
- [ ] A fistula or MPDSR record displays its patient name correctly
- [ ] `rotate_fernet_key` run, after verification

Cutover

- [ ] All fourteen forms imported and deployed in your own Kobo account
- [ ] A REST Service added to each one, pointing at your domain
- [ ] New asset UIDs put in `.env` (9d), especially the two write-back ones
- [ ] `sync_kobo_uids --apply` run, all fourteen forms matched (9e)
- [ ] The two lookup files exported and attached to CIPRB-1 and CIPRB-10 (9f)
- [ ] CIPRB-1 opened in Enketo and the patient dropdown is populated
- [ ] A real test submission arrives and appears in the approval queue
- [ ] The test record is approved, then deleted

After

- [ ] Both cron jobs installed and each run once by hand
- [ ] Daily database backup running
- [ ] `FERNET_KEY` and the media volume backed up separately
- [ ] A restore tested into a scratch database
- [ ] The three settings in Step 13 pointed at your own domain
- [ ] Previous server kept running for one week, in case a form was missed
- [ ] Named person responsible for backups, certificate renewal and key custody

---

Five questions for CIPRB IT

1. Can the server have a public domain and TLS certificate, with ports 80 and 443 reachable from outside? If not, the system cannot receive field data at all, and nothing else in this guide matters.
2. Who holds `FERNET_KEY` and `SECRET_KEY` after the handover, and where are they kept?
3. Who takes the backups, and who has tested restoring one?
4. Docker, or installed directly on the machine?
5. Who is responsible when the TLS certificate fails to renew?